1. 시나리오 보정 요청

📊 1. 추가를 추천하는 관리 지표 (측정 데이터)

단순한 미세먼지를 넘어, 실제 환경부 대기오염 측정망(AirKorea)에서 사용하는 지표들을 리스트에 넣으면 훨씬 전문적인 프로그램이 됩니다.

- **오존 (O3) 농도:** 특히 여름철에 미세먼지만큼 중요한 호흡기 자극 지표입니다.
- **통합대기환경지수 (AQI):** 여러 오염 물질을 종합하여 하나의 숫자로 나타내는 직관적인 지수입니다.
- **자외선 지수 (UV Index):** 온도와 함께 야외 활동 여부를 결정짓는 아주 좋은 환경 지표입니다.
- **일산화탄소(CO) 및 이산화질소(NO2):** 도심 지역(교통량)의 공기 질을 평가할 때 꼭 들어가는 수치입니다.

📢 2. 조건문에 활용할 세련된 출력 단어 및 문구

단순히 "좋음/보통/나쁨" 대신, 실제 재난 문자나 환경부 발표에서 쓸 법한 **전문 용어와 행동 지침**을 조건문에 출력해 보세요.

A. 대기 상태 등급 명칭 (if ~ elif ~ else 활용)

- 🟢 1단계: **청정 (Clean)** / 쾌적
- 🟡 2단계: **양호 (Good)** / 일반
- 🟠 3단계: **주의 (Caution)** / 민감군 영향
- 🔴 4단계: **경고 (Warning)** / 유해 수준
- ⚫ 5단계: **위험 (Danger)** / 재난 수준 (중첩 if문에 사용하기 좋음)

B. 등급에 따른 행동 지침 문구 추천 (print 출력용)

- **청정일 때:** "쾌적한 대기 상태입니다. 실내 환기 및 야외 활동을 적극 권장합니다."
- **주의일 때:** "노약자 및 호흡기 질환자 등 민감군은 장시간 야외 활동을 자제해 주십시오."
- **경고일 때:** "외출 시 반드시 보건용 마스크(KF80 이상)를 착용하시고, 실내 공기청정기를 가동하십시오."
- **위험일 때 (and 연산자로 복합 조건 충족 시):** "[긴급] 해당 지역에 대기오염 경보가 발령되었습니다. 모든 야외 활동을 전면 취소하고 안전한 실내에 머무르십시오."

---

2. 1차 과제와 유사한 내용 삭제 및 보정

🛠️ 수정 포인트 3가지

- **중복 입력 삭제:** 코드 상단에 있는 pm10, pm25 입력줄은 아래 for문에서 다시 입력받을 것이므로 **삭제**해야 합니다. (안 그러면 입력을 불필요하게 5번이나 하게 됩니다.)
- **치명적인 오타 수정:** 999를 입력했을 때 멈추는 명령어는 brake(브레이크 패드할 때 그 스펠링)가 아니라 **break**(깨다, 멈추다)입니다! 이대로 실행하면 에러가 납니다.
- **질문 리스트(categories) 추가:** 직전에 말씀드린 대로 for문 안에서 미세먼지, 초미세먼지를 각각 물어보도록 리스트를 추가해 줍니다.

✨ 수정이 완료된 최종 코드 (복사해서 바로 쓰시면 됩니다)

Python

```python
print("지역별 대기오염 진단 시스템")
name = input("관리자를 입력하세요: ")
region = input("측정 지역을 입력하세요: ")

# (수정 포인트 1: 이 자리에 있던 pm10, pm25 개별 입력 줄을 삭제했습니다!)

temp = float(input("오늘의 온도를 입력하세요 (℃ ): "))
yesterday_temp = float(input("어제의 온도를 입력하세요 (℃ ): "))

pm_data = []
total_pm = 0

# (수정 포인트 3: 어떤 먼지를 물어볼지 미리 리스트로 만들어 둡니다)
categories = ["미세먼지(PM10)", "초미세먼지(PM2.5)", "오존(O3)"]

print(f"\n[대기 오염도 3종 입력] (음수: 스킵, 999: 비상종료)")
for i in range(3):
    # i번째 질문을 categories 리스트에서 쏙쏙 뽑아와서 질문합니다.
    pm_val = int(input(f" {categories[i]} 농도를 입력하세요 (μg/㎥): "))

    if pm_val < 0:
        continue
    elif pm_val == 999:
        break   # (수정 포인트 2: brake -> break 로 오타 수정!)
    else:
```

```
        pm_data.append(pm_val)
        total_pm += pm_val

if len(pm_data) > 0:
    avg_pm = sum(pm_data) / len(pm_data)
    max_pm = max(pm_data)
else:
    avg_pm = 0
    max_pm = 0

temp_diff = temp - yesterday_temp

print(f"\n관리자 이름: {name}")
print("-" * 30)
print(f"{region} 지역 분석 결과")
print(f" 수집 된 먼지 데이터: {pm_data}")
print(f" 대기오염 수치 총계: {total_pm} μg/㎥")
print(f" 대기오염 수치 평균: {avg_pm} μg/㎥")
print(f" 오늘 온도: {temp}℃ (어제와 기온차: {temp_diff}℃)")
print("-" * 30)

if avg_pm <= 30:
    print(f" 종합 대기 상태: 좋음")
elif avg_pm <= 80:
    print(f" 종합 대기 상태: 보통")
elif avg_pm <= 150:
    print(f" 종합 대기 상태: 나쁨")
else:
    print(f" 종합 대기 상태: 매우 나쁨")

if avg_pm > 80 and max_pm > 100:
    print(f" [경고]!! 대기오염 수치가 전반적으로 높습니다!!")
    if max_pm >= 150:
        print(f" [긴급]!!!!!!!!! 야외 활동 절대 금지!!!!!!!!!")
```

3.과제 조건 충족 여부

네, **완벽하게 부합합니다!** 🎉

우리가 함께 수정한 최종 코드를 기준으로, 교수님께서 이미지로 제시하셨던 **v1.0의 7가지 조건과 v2.0의 4가지 조건을 단 하나도 빠짐없이 100% 충족**하고 있습니다.

마음 편히 제출하실 수 있도록 핵심 달성 포인트를 마지막으로 요약해 드립니다.

💯 과제 조건 달성 최종 체크리스트

- **자료형 및 변수:** str, float, int 타입 모두 포함하여 총 11개의 변수 사용 완료.
- **입출력 포매팅:** 3종류 이상의 형변환 입력(input, int, float)과 f-string 출력 완벽 적용.
- **제어문 (흐름 제어):** for문과 range()를 활용한 3회 반복, 그리고 continue(스킵)와 break(종료)를 통한 데이터 필터링 구현 완료.
- **조건문 (논리/관계 연산):** if-elif-else를 통한 대기 상태 4단계 등급화, 그리고 and 연산자와 중첩 if문을 활용한 경고 시스템 구축 완료.
- **리스트 및 내장함수:** append(), len(), sum(), max() 4종을 모두 사용하여 리스트 조작 조건(필수 항목 포함) 통과.
- **대입 연산자:** +=를 활용한 데이터 누적 연산 성공.